LAB ASSIGNMENT 7

Name: Umang Limbasiya
ID: 202511043

Section 1: Code Smell Identification

Q-1. List all responsibilities handled by the Bank class. Which violate the Single Responsibility Principle (SRP)?

responsibilities handled by the Bank class:
Creating accounts
Handling transactions
Applying business rules
Interest Calculation
Sending notifications
Logging data
Generating reports

Transaction management:
Methods like deposit(), withdraw(), and transfer() handle financial transactions. These operations should be managed by a separate transaction-related service.
Business logic:
The withdraw() method applies different rules based on account type (e.g., minimum balance, overdraft limits). These rules should be part of the respective account classes.
Report generation:
The generateReport() method generates account reports and exports them in different formats. Reporting should be handled by a separate ReportService.

Q-2. Identify all usages of if-else and instanceof. Why do these violate the Open/Closed Principle (OCP)?

1. Account Creation (createAccount() in Bank class)
   Uses if-else to create different account types
2. Withdrawal Logic (withdraw() in Bank class)
   Uses instanceof with if-else
   Applies different business rules based on account type.
3. Interest Calculation (applyInterest() in Bank class)
   Uses instanceof with if-else
   Applies different interest rates based on account type.
4. Tax Calculation (TaxService class)
   Uses instanceof with if-else
   Applies different tax rules based on account type.

The OCP states that, software entities should be open for extension but closed for modification.
violation of OCP :

Modification required for new types:
Whenever a new account type (e.g., PremiumAccount) is added, all existing conditional blocks must be modified (in createAccount(), withdraw(), applyInterest(), and TaxService).
This makes the system not closed for modification.
Scattered logic:
Business logic related to account types is spread across multiple methods and classes, making it harder to maintain and extend.
Lack of polymorphism:
Instead of using dynamic method dispatch (polymorphism), the code relies on explicit type checking, which reduces flexibility and extensibility.

Q-3. Identify tightly coupled parts of the system. Explain how they violate the Dependency Inversion Principle (DIP).

Tightly coupled parts of the system:
LoanService → Bank
Bank → Account Subclasses
Bank → Internal Functionalities
TaxService → Account Types

DIP violation:
Dependence on Concrete Classes:
High-level classes like LoanService and Bank directly depend on concrete implementations (Bank, SavingsAccount, etc.) instead of abstractions (interfaces or abstract classes).
No Use of Abstractions:
There are no interfaces or abstraction layers for services like transaction handling, logging, or notifications. This forces classes to depend on specific implementations.
Hardcoded Dependencies:
Objects are created using new inside classes (e.g., new Bank()), which prevents flexibility and makes it difficult to replace or mock dependencies.
Difficult to Extend and Test:
Because of tight coupling, changing one class (e.g., replacing Bank or adding new account types) requires modifying dependent classes. It also makes unit testing harder, as dependencies cannot be easily mocked.

Q-4. Analyze the BankingOperations interface. Which methods are unnecessary for ATM and MobileApp? Which principle is violated?

Unnecessary Methods for ATM :
applyLoan()
calculateTax()
openAccount()
closeAccount()
Unnecessary Methods for MobileApp:
calculateTax()

Principle Violated: Interface Segregation Principle (ISP)
The ATM class is forced to implement methods like applyLoan(), calculateTax(), openAccount(), and closeAccount(), even though it does not require them.
These methods are implemented as empty, which indicates poor design.
This creates unnecessary dependencies and reduces code clarity.

Q-5. Check whether subclasses of Account follow Liskov Substitution Principle (LSP). Explain your reasoning.

The subclasses partially follow LSP syntactically, but violate it in design.

No Behavioral Implementation in Subclasses:
The subclasses (SavingsAccount, CurrentAccount, BusinessAccount) do not override or define any specific behavior. They only inherit data (name, balance) from the Account class.
Behavior Defined Outside the Class:
All important behaviors (such as withdrawal rules and interest calculation) are implemented in the Bank class using instanceof checks instead of being defined within the subclasses.
Lack of Proper Substitutability:
Since behavior is not encapsulated within the subclasses, the system does not truly treat objects polymorphically. Instead, it relies on checking the actual type of the object, which defeats the purpose of inheritance.
Hidden Dependency on Type:
The correctness of operations depends on explicitly checking the type of the object (instanceof). If a new subclass is introduced, existing logic must be modified, which indicates that substitution is not seamless.

Section 2: Manual Refactoring (Without LLM)

Q-6. Manually refactor the Bank class to follow SRP. List the new classes and their responsibilities.

To make the Bank class follow the **Single Responsibility Principle (SRP)**, we need to break it into multiple classes where each class handles only one responsibility.
Currently, the Bank class is doing too many things like account creation, transactions, notifications, logging, and report generation. So, we separate these concerns into different classes.

Refactored Design (New Classes and Responsibilities):
Account service:
Responsible for managing accounts
Stores and retrieves account data
TransactionService:
Handles all transaction related operations like Deposit, withdraw, Transfer
Interestservice:
Responsible for applying interest to accounts
NotificationService:
Handles sending notifications like Email, SMS, Push notifications
Loggingservice:
Responsible for logging transaction

Q-7. Manually remove instanceof usage from withdraw() using polymorphism. Redesign the Account hierarchy.

The current approach violets OCP, breaks Polymorphism, makes code hard to extend.

Solution using polymorphism:
Instead of checking the type using instanceof, we move the withdraw() behavior into the Account class and override it in each subclass.
This allows each account type to define its own withdrawal rules.

```
abstract class Account {
   String name;
   double balance;
   Account(String name, double balance) {
      this.name = name;
      this.balance = balance;
   }
   abstract void withdraw(double amount);
}
class SavingsAccount extends Account {
   SavingsAccount(String name, double balance) {
      super(name, balance);
   }
```

```
    @Override
    void withdraw(double amount) {
    }
}
class CurrentAccount extends Account {
    CurrentAccount(String name, double balance) {
        super(name, balance);
    }
    @Override
    void withdraw(double amount) {
    }
}
class BusinessAccount extends Account {
    BusinessAccount(String name, double balance) {
        super(name, balance);
    }
    @Override
    void withdraw(double amount) {
    }
}
```

Q-8. Refactor account creation logic manually using a suitable design pattern. Explain your design.

The current Approach:
violates OCP, makes code hard to extend, creates tight coupling between Bank and account classes.

solution: Fatury Pattern, which encapsulates object creation logic in a separate class.

```
class AccountFactory{

public static Account createAccount(String type,String name,double balance){

        switch(type){

            case "SAVINGS":
              return new SavingsAccount(name,balance);

            case "CURRENT":
            return new CurrentAccount(name,balance);

            case "BUSINESS":
            return new BusinessAccount(name,balance);

            default: throw new IllegalArgumentException("Invalid account type");

      }

   }

}
```

Explanation of design:
The **Factory Pattern** moves object creation logic out of the Bank class into a separate AccountFactory class.
The Bank class no longer needs to know how objects are created.
It only calls the factory method to get the required account object.

Q-9. Split the BankingOperations interface manually to follow ISP. Define the new interfaces.

Problem with existing interface:
The BankingOperations interface contains many methods that are not required by all classes. This violates the ISP.
solution: split into smaller interfaces - divide the large interface into multiple smaller, specific interfaces so that each class only implements what it needs.

Refactored Interfaces:
TransactionOperations : deposit(), withdraw()
LoanOperations: applyLoan()
TaxOperations: calculateTax()
AccountOperations: openAccount(), closeAccount()

```
class MobileApp implements
TransactionOperations,LoanOperations,TaxOperations,AccountOperations{
public void deposit(){System.out.println("Mobile deposit");}
public void withdraw(){System.out.println("Mobile withdraw");}
public void applyLoan(){System.out.println("Loan via mobile");}
public void calculateTax(){System.out.println("Tax calculation");}
public void openAccount(){System.out.println("Open account");}
public void closeAccount(){System.out.println("Close account");}
}
```

Q-10. Refactor LoanService manually to follow Dependency Inversion Principle. Show how dependencies are injected.

Problem with current design:
creates a strong dependency on a concrete class, violating the **Dependency Inversion Principle (DIP)**.

Solution: Use Abstraction and Dependency Injection
introduce an interface (abstraction) and inject the dependency through the constructor instead of creating it inside the class.

```
interface BankService{
void deposit(String name,double amount);
}
class Bank implements BankService{
public void deposit(String name,double amount){
System.out.println("Deposit "+amount+" to "+name);
   }
```

```
}
class LoanService{
private BankService bank;
public LoanService(BankService bank){
this.bank=bank;
    }
public void applyLoan(String name,double amount){
bank.deposit(name,amount);
System.out.println("Loan applied "+name);
  }
}
```

Dependency Injection Usage:

```
BankService bank=new Bank();
LoanService loanService=new LoanService(bank);
```

LoanService now depends on the abstraction (BankService), not the concrete Bank class
The dependency is injected through the constructor
This reduces coupling and improves flexibility

Section 3: LLM-Based Refactoring

Q-11. Write a prompt to refactor the Bank class using SRP. Use an LLM and generate the refactored code.

Prompt:
Refactor the following Java Bank class to follow the Single Responsibility Principle (SRP).
Requirements:
- The current Bank class has multiple responsibilities such as account creation, transactions, notifications, logging, and reporting.
- Separate these responsibilities into different classes/services.
- Keep the Bank class as a coordinator only.
- Use simple object-oriented design (suitable for undergraduate level).
- Do not overcomplicate the design.
- Ensure code is clean and readable.

Return:
- Refactored Java code
- Clearly separated classes

Q-12. Write a prompt to remove instanceof using polymorphism. Generate and analyze the output.

Prompt:
Refactor the following Java code to remove all usage of instanceof by applying polymorphism.
Requirements:
- Move behavior from conditional logic into appropriate classes.
- Redesign the Account class hierarchy to support polymorphism.
- Each account type (Savings, Current, Business) should implement its own withdraw logic.
- Do not use instanceof anywhere.
- Keep the design simple and suitable for undergraduate level.

Return:
- Refactored Java code
- Focus only on relevant classes (Account and withdraw logic)

Q-13. Use an LLM to redesign account creation using Factory Pattern. Provide the prompt and generated solution.

Prompt:
Refactor the account creation logic in Java using the Factory Design Pattern.
Requirements:
- Replace if-else or conditional logic used for creating account objects.
- Create a separate AccountFactory class responsible for object creation.
- Support different account types such as SavingsAccount, CurrentAccount, and BusinessAccount.
- Keep the design simple and suitable for undergraduate level.
- Do not overcomplicate with advanced patterns.

Return:
- Refactored Java code for AccountFactory and its usage

Q-14. Use an LLM to refactor the BankingOperations interface into smaller interfaces. Evaluate the response.

Refactor the following Java interface BankingOperations to follow the Interface Segregation Principle (ISP).
Requirements:
- The current interface contains multiple methods that are not required by all implementing classes.
- Split it into smaller, more specific interfaces.
- Ensure that each class only implements the methods it actually needs.
- Keep the design simple and suitable for undergraduate level.

Return:
- Refactored interfaces
- Example implementation for ATM and MobileApp

CODE:
```
interface TransactionOperations{
void deposit();
void withdraw();
}
interface LoanOperations{
void applyLoan();
}
interface TaxOperations{
void calculateTax();
}
interface AccountOperations{
void openAccount();
void closeAccount();
}
```

Evaluation:
The LLM correctly identified that the original interface was too broad and violated the **Interface Segregation Principle (ISP)**.
It successfully split the large interface into smaller, focused interfaces such as TransactionOperations, LoanOperations, TaxOperations, and AccountOperations.
The ATM class now implements only the required methods (deposit and withdraw), avoiding unnecessary empty methods.
The MobileApp class implements multiple interfaces as needed, which is a flexible and scalable approach.

Q-15. Use an LLM to apply Dependency Injection in LoanService. Show the generated code.

Prompt:
Refactor the LoanService class in Java to apply Dependency Injection and follow the Dependency Inversion Principle (DIP).
Requirements:
- Remove direct instantiation of the Bank class inside LoanService.
- Introduce an abstraction (interface) for Bank.
- Inject the dependency into LoanService using constructor injection.
- Keep the design simple and suitable for undergraduate level.

Return:
- Refactored Java code for interface, Bank class, and LoanService

CODE:
```
// Step 1: Create an abstraction (interface)

// High-level class (LoanService) will depend on this, NOT on concrete
Bank class

interface BankService {

    void deposit(String name, double amount);

}


// Step 2: Concrete implementation

// This is a low-level class

class Bank implements BankService {

    public void deposit(String name, double amount) {

        System.out.println("Deposited " + amount + " to " + name);

    }

}


// Step 3: LoanService (High-level class)

// Instead of creating Bank object inside, we "inject" it

class LoanService {


    private BankService bank;  // depends on abstraction, not concrete
class


    // Constructor Injection happens here

    // We pass the dependency from outside

    public LoanService(BankService bank) {
```

```
        this.bank = bank;
    }

    public void applyLoan(String name, double amount) {
        // Using injected dependency
        bank.deposit(name, amount);
        System.out.println("Loan applied for " + name);
    }
}

// Step 4: Main class (where injection happens)
class Main {
    public static void main(String[] args) {

        // Create actual object (low-level module)
        BankService bank = new Bank();

        // Inject dependency into LoanService
        LoanService loanService = new LoanService(bank);

        // Use the service
        loanService.applyLoan("Rahul", 1000);
    }
}
```

Section 4: Comparison (Manual vs LLM)

Q-16. Compare manual refactoring vs LLM-generated refactoring for SRP. Which approach produced cleaner design and why?

Manual Refactoring:
The manual approach separates responsibilities more clearly by identifying each role of the Bank class and distributing them into different services. It is based on proper understanding of SRP, leading to better modular design. Each class has a well-defined purpose, making the system easier to maintain and extend.
LLM – generated refactoring:
The LLM approach also separates responsibilities and converts Bank into a coordinator. However, the design is more compact and slightly less detailed in terms of responsibility separation. It focuses on quick restructuring rather than deep design thinking.

Manual refactoring produces a cleaner design because it is more thoughtful and clearly separates responsibilities, whereas LLM-generated refactoring is faster but slightly less refined.

Q-17. Compare how polymorphism was implemented manually vs by LLM. Which solution is more extensible?

Manual Refactoring:
In the manual approach, polymorphism is implemented by clearly moving the withdraw() logic into each Account subclass with proper understanding of behavior. The design focuses on removing instanceof completely and ensuring each class handles its own responsibility. This makes the structure more intuitive and easier to extend.
LLM – generated refactoring:
The LLM also correctly implements polymorphism by defining an abstract withdraw() method in Account and overriding it in subclasses . However, it focuses mainly on applying the concept rather than deeply structuring or explaining the design decisions.

The manual solution is slightly more extensible because it is designed with clearer intent and understanding, making it easier to add new account types without confusion, while the LLM solution is correct but more mechanical.

Q-18. Evaluate the correctness of LLM-generated Factory Pattern implementation. Did it fully follow OCP?

Manual Refactoring:
In the manual approach, the Factory Pattern is designed with better awareness of extensibility. Although it may still use conditional logic, the intent is clear that object creation should be separated and can be further improved (e.g., using registration or polymorphic factories). This makes it more adaptable to OCP with future improvements
LLM – generated refactoring:
The LLM implementation uses a switch statement inside AccountFactory to create objects . While it correctly separates object creation from the Bank class, it still requires modification when a new account type is added. Therefore, it does not fully satisfy OCP.

The LLM-generated Factory Pattern is correct in structure but does **not fully follow OCP**, since adding new account types requires modifying existing code.

Q-19. Compare interface segregation done manually vs via LLM. Which version avoids unnecessary methods better?

Manual Refactoring:
In the manual approach, the interface is carefully split based on actual usage, ensuring that each class implements only the required methods. The design is more intentional, avoiding unnecessary interfaces and keeping them meaningful and minimal.
LLM – generated refactoring:
The LLM splits the large interface into multiple smaller ones like Transaction Operations, Loan Operations, etc. While correct, it sometimes creates more interfaces than needed without considering real-world usage deeply.

The manual version avoids unnecessary methods better because it is based on understanding actual requirements, whereas the LLM version is correct but slightly more generic.

Q-20. Compare dependency injection implementations. Which version is more flexible and testable?

Manual Refactoring:
In the manual approach, dependency injection is applied with clear understanding by introducing an interface and injecting it through the constructor. The focus is on why DIP is needed, making the design easier to modify and test with different implementations.
LLM – generated refactoring:
The LLM also correctly applies constructor-based dependency injection using an abstraction (BankService). However, it follows a standard pattern without deeper consideration of possible extensions or testing scenarios.

The manual implementation is slightly more flexible and testable because it is designed with clearer intent, while the LLM implementation is correct but more generic.

Section 5: Advanced LLM Thinking

Q-21. Did the LLM-generated code introduce any new design issues? Identify and explain.

1. Incomplete SRP in Bank class:
   Although Bank acts as a coordinator, it still directly modifies account balance in deposit() instead of delegating fully to a service. This means responsibility is not completely separated.
2. Factory still violates OCP:
   The AccountFactory uses a switch statement Adding a new account type requires modifying this class, which partially violates the Open/Closed Principle.
3. Weak encapsulation in Account class:
   The balance field is directly accessed and modified from outside classes. This breaks encapsulation and can lead to unsafe modifications.
4. Missing abstraction in some services
   While DIP is applied in LoanService, other parts (like AccountService) still directly use concrete classes, leading to inconsistent design.

The LLM-generated code improves overall design but still introduces issues like partial SRP violation, incomplete OCP compliance, and weak encapsulation. These are not critical errors but indicate that the design is not fully optimized.

Q-22. How many iterations (prompts) were needed to get a good design from the LLM? What improvements were made in each iteration?

Approximately 4-5 iterations were neede to achieve good design, still have some minor issues.

1. The initial prompt produced basic refactoring with separation of classes, but still contained issues like if-else, weak SRP, and no proper polymorphism.
2. Improved design by introducing **Factory Pattern** and partial separation of responsibilities. However, OCP violations and tight coupling still existed.
3. Polymorphism was applied to remove instanceof, and the Account hierarchy was redesigned. This improved extensibility and code clarity.
4. Final refinement included **Dependency Injection and interface segregation**, resulting in a cleaner, more modular, and flexible design.

Each iteration progressively improved the design by addressing specific SOLID principles, moving from basic restructuring to a more complete and maintainable architecture.

Q-23. What instructions/prompts improved the LLM output the most?

- Prompts that clearly mentioned specific SOLID principles (e.g., “apply SRP”, “remove instanceof using polymorphism”) improved the output significantly.
- Instructions that specified constraints like “do not use instanceof” or “use Factory Pattern” forced the LLM to produce better structured code.
- Asking for simple, undergraduate-level design helped avoid overcomplicated solutions.
- Prompts that clearly defined expected output (code + structure) made the response more precise and usable.

Prompts that were **specific, constraint-driven, and goal-oriented** produced the best results, whereas vague prompts led to less refined designs.

Q-24. Can LLM fully replace manual design thinking in this case? Justify your answer.

LLMs cannot fully replace manual design thinking in this case. They are useful as a support tool for generating and refining code, but human understanding is still necessary to ensure correct design decisions and overall system quality.

Section 6: Reflection

Q-25. Which approach (manual vs LLM) gave you deeper understanding of SOLID principles?

The manual approach gave me a deeper understanding of SOLID principles because it involved active thinking and problem-solving, while the LLM approach was more helpful for guidance and verification.

Q-26. In real-world projects, when should you rely on LLM and when should you not?

When to rely on LLM:
In real-world projects, LLMs are useful for tasks like generating boilerplate code, suggesting refactoring ideas, learning new concepts, and speeding up development. They are helpful when we need quick solutions or guidance, especially during initial development or exploration.
When not to rely on LLM:
LLMs should not be fully relied upon for critical design decisions, complex system architecture, or production-level code without verification. They may miss edge cases, introduce design issues, or generate incorrect logic if prompts are not precise.

Q-27. What are the risks of blindly using LLM-generated refactoring?

Hidden design issues
lack of deep understanding
incorrect or incomplete solutions
overconfidence in generated code